

Morpion à cases bloquées

Étude comparative de MinMax et du Q-learning

Q-learning

MinMax

Cases bloquées

X	O	#
	X	
#		O

NKEMLA LIENOU Steve Frede
RICHETER Marie

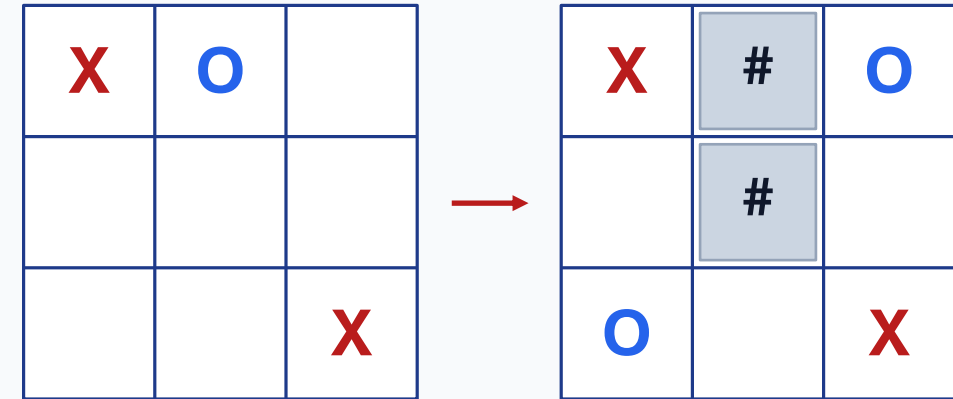
Notre idée de départ

Partir d'un jeu simple pour observer le comportement de deux IA différentes.

PROBLÈME

Comment une contrainte simple change-t-elle la stratégie ?

Le morpion classique est petit et entièrement observable.
Les cases bloquées diminuent les chemins de victoire.
L'objectif est d'observer l'adaptation de deux IA.



Morpion classique

Morpion contraint

MinMax : raisonnement par arbre

Q-learning : apprentissage par essais

La réflexion sur les cases bloquées

Une amélioration proposée après la remarque du professeur.

CONCEPTION

Problème : trop de blocage peut rendre le jeu stérile.

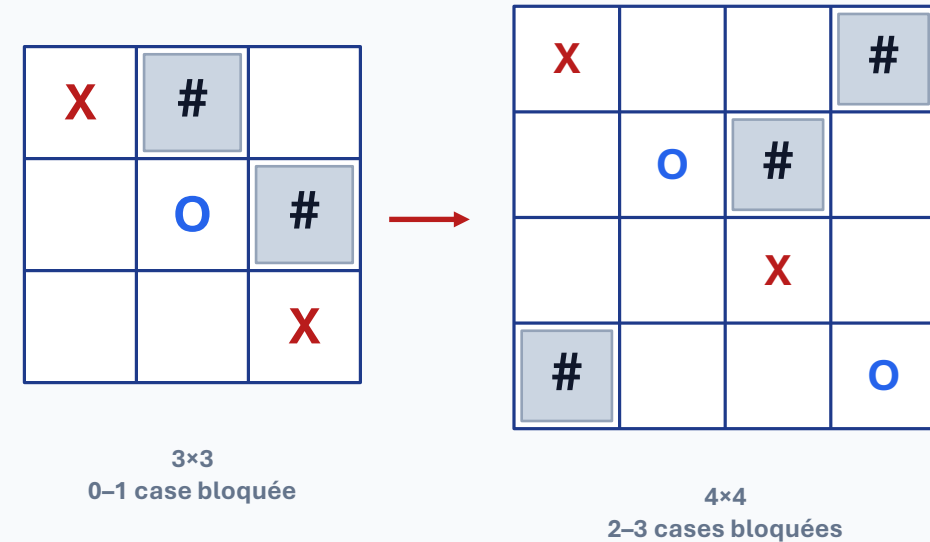
Sur une grille 3×3, deux cases bloquées peuvent casser plusieurs lignes gagnantes.

Le risque de nul augmente fortement.

Nous avons donc contrôlé les configurations trop fermées.

Décision de conception

Quand le blocage augmente, la taille du plateau peut augmenter pour garder un espace stratégique suffisant.



$\geq 2/3$

Lignes ouvertes minimales contrôlées

moins de nuls

Objectif

Démarche scientifique

Nous avons structuré le projet comme une expérimentation.



Observer

Les cases bloquées changent
les lignes de victoire.



Proposer

Comparer Q-learning et MinMax.



Implémenter

Créer le moteur du jeu, les IA et
les tests.



Analyser

Mesurer victoires, nuls, limites
et temps de calcul.

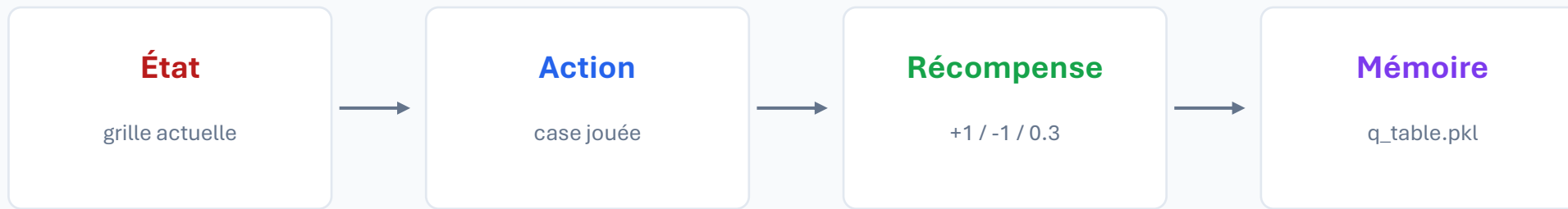
Le projet ne se limite pas à coder : il vise à comprendre comment les contraintes modifient le comportement des algorithmes.

Q-learning : apprendre par expérience

L'agent améliore ses choix en jouant de nombreuses parties.

IA #1

État → Action → Récompense → Nouvelle valeur Q



Dans notre projet

X = agent Q-learning • O = adversaire • entraînement contre un joueur aléatoire

X		O
#	X	
		O

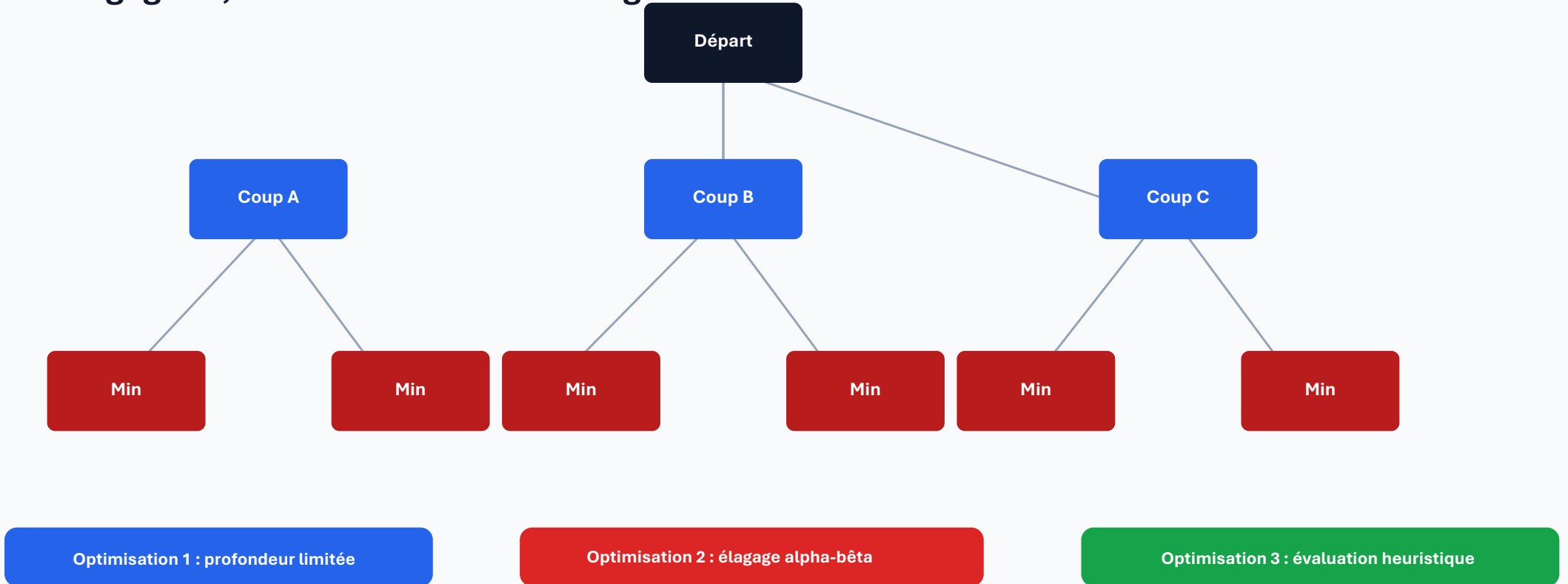
Une situation de jeu devient une chaîne de caractères utilisable par l'IA.

MinMax : raisonner par anticipation

L'IA explore les futurs coups possibles et choisit le meilleur scénario.

IA #2

Max cherche à gagner ; Min cherche à limiter le gain de Max.



Architecture du projet

Chaque fichier a un rôle précis pour rendre le projet lisible et testable.

morpion.py

moteur du jeu

qlearning.py

agent Q-learning

entrainement.py

apprentissage

minMax.py

IA MinMax optimisée

evaluation.py

comparaison IA

main.py

menu & démo

Flux principal : main.py → morpion.py → IA choisie → résultats → evaluation.py

Évolution du projet

Nous avons amélioré le projet par étapes à partir des tests et remarques.



Message central

Le projet montre une progression : d'un simple morpion vers une étude algorithmique avec contraintes, apprentissage et optimisation.

Expérimentations et résultats observés

L'entraînement valide que le Q-learning apprend une politique exploitable.

RÉSULTATS

10 000

Parties d'entraînement

4 883

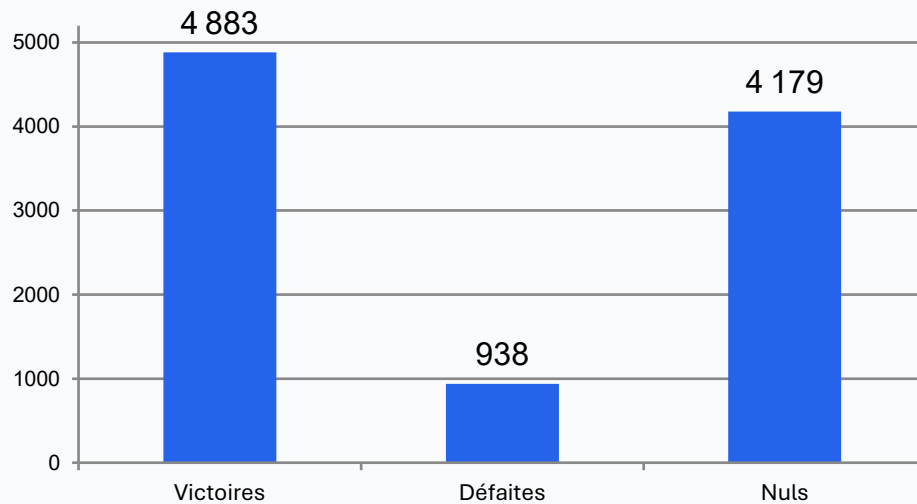
Victoires Q-learning

938

Défaites

4 179

Matches nuls



Lecture des résultats

L'agent perd peu contre un adversaire aléatoire.
Le nombre de nuls reste important : c'est normal dans un morpion contraint.
La Q-table permet de réutiliser l'apprentissage sans réentraîner.

Analyse comparative

Deux IA, deux logiques et deux limites différentes.

MinMax

Raisonne en explorant les coups futurs.
Très fort sur une petite grille.
Deviens coûteux quand la taille augmente.
Besoin d'optimisation : alpha-bêta.

Q-learning

Apprend par répétition et récompenses.
Réutilise sa mémoire `q_table.pkl`.
Dépend de la qualité de l'entraînement.
Peut s'adapter à des configurations variées.

Conclusion : MinMax explique bien la décision, Q-learning montre l'apprentissage par expérience.

Démonstration du projet

Une séquence simple, fluide et visuelle pour présenter rapidement le fonctionnement du projet.

Objectif de la démo

Montrer l'interface, lancer un mode, afficher les cases bloquées et observer un tour d'IA.

1

Lancer l'interface

Ouvrir interface.py puis afficher la page d'accueil.

Page d'accueil

2

Choisir un mode

Sélectionner Humain vs IA ou la comparaison des IA.

Mode de jeu

3

Montrer le plateau

Expliquer les cases bloquées et la taille adaptative du plateau.

Cases bloquées

4

Observer l'IA

Montrer un tour joué par Q-learning ou MinMax et commenter la logique.

Décision IA

Conseils



Préparer q_table.pkl à l'avance



Privilégier une démo courte



Avoir une capture de secours

Conclusion

Ce que nous retenons du projet.

Nous avons transformé un jeu simple en terrain d'expérimentation IA.

- 1 Les cases bloquées changent réellement la stratégie.
- 2 Q-learning apprend par expérience et conserve une mémoire.
- 3 MinMax est puissant, mais doit être optimisé sur des plateaux plus grands.
- 4 La comparaison montre deux visions différentes de l'IA.

Ouverture : améliorer l'entraînement, enrichir l'heuristique, tester d'autres configurations.